PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Project Overview 3
2. Background & Business Objective 3
3. Product Purpose 4
3.1. Main Goal of the System 4
3.2. Intended Users (Target Audience) 4
3.3 Success Metrics 4
4. System Functionalities 5
4.1. Description 5
4.2. Key Functionalities 5
4.3. AI Model & Prompt Design 9
5. User Stories & Use Cases 14
6. Features Included (Scope Definition) 14
7. Features Not Included (Scope Control) 15
8. Assumptions & Constraints 15
9. Risks & Questions Throughout Development 15

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: InvAgent (Inventory Procurement & Management System)
Version: 1.0.0

1. Project Overview

- **Problem Statement**: Small to medium enterprises (SMEs) in Malaysia rely on fragmented, manual workflows for inventory management—spreadsheets for stock, WhatsApp for orders, email for approvals, and separate accounting tools. This results in stockouts, overstocking, delayed procurement, and human error. Existing ERP systems are too expensive and rigid.
- **Target Domain**: Operational workflow automation for procurement and inventory management in retail and logistics SMEs.
- **Proposed Solution Summary**: InvAgent is an AI-powered workflow engine where Z. AI's GLM acts as the central reasoning agent. It listens to unstructured inputs (e.g., "We're almost out of rice bags"), orchestrates existing API endpoints (inventory CRUD, supplier portals, procurement triggers), and executes multi-step procurement workflows—from request to order to tracking—without human intervention unless necessary.
- **Technical Foundation:** Built on the existing Spring Boot inventory management system with:
  - InventoryController - Stock management endpoints
  - ProcurementController - Restock orchestration with scheduled automation
  - SupplierController - Supplier discovery and selection
  - JPA entities: Product, Supplier, ProductSupplier, Order, InventoryLog

2. Background & Business Objective

- **Background of the Problem:**

Currently, procurement involves: manual stock check → human decision to reorder → email/call supplier → wait for quote → approval → purchase order → delivery tracking. Each step requires switching between tools and human judgment. When data is missing (e.g., no stock count), workflows stop entirely.

- **Importance of Solving This Issue:**

Automating this reduces stockouts by 60%, cuts procurement cycle time from days to minutes, and lowers operational costs. It also democratizes inventory management—non-technical staff can trigger complex workflows via natural language.

- **Strategic Fit / Impact:**

Aligns with the challenge's focus on GLM as a central reasoning engine. InvAgent is not a simple chatbot; if GLM is removed, the system cannot reason about ambiguity, decide reorder quantities, or orchestrate APIs. It demonstrates stateful, adaptive workflow automation.

© UMHackathon 2026
Email: umhackathon@um.edu.my

3. Product Purpose

3.1. Main Goal of the System:

To transform manual, fragmented inventory procurement into an intelligent, adaptive, and automated workflow powered by GLM.

3.2. Intended Users (Target Audience):

Role	Primary Actions
Warehouse Supervisors	Monitor stock, trigger urgent restocks
Procurement Officers	Approve high-value orders, manage suppliers
Small Retail Business Owners	View dashboard, get low-stock alerts
Operations Managers	Configure auto-restock rules, review escalation logs

3.3 Success Metrics

- Reduce stockouts by 60%
- Cut procurement cycle time from 2 days → 5 minutes
- Achieve 95% accurate tool/API selection by GLM
- Zero critical failures without a human escalation path

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. System Functionalities

4.1. Description

InvAgent operates as a stateful workflow engine that bridges natural language communication with the existing Spring Boot inventory management system. It receives unstructured inputs (voice/text "Urgent: Need 50kg flour"), reasons across stages (check stock → select supplier → get approval → create PO), and generates structured actions that map directly to the existing API endpoints:

API Endpoint	InvAgent Action
`POST /inventory/add`	Execute restock after approval
`POST /inventory/remove`	Record sales/usage from natural language
`GET /inventory/restock-check`	Evaluate reorder necessity
`GET /suppliers/{productId}`	Discover supplier options
`GET /suppliers/{productId}/best`	Select optimal supplier
`POST /procurement/restock/{productId}`	Trigger manual restock
`@Scheduled autoRestock()`	Automated 10-minute health check

The system maintains workflow state across multiple turns (e.g., `awaiting_stock_check` → `awaiting_supplier_selection` → `awaiting_approval` → `completed`), ensuring that interrupted workflows can resume without losing context.

4.2. Key Functionalities:

- Unstructured Intake: Parses messages, emails, voice transcripts, or scanned forms. The GLM extracts:
  - Intent: restock_request, stock_query, supplier_inquiry, order_status
  - Entities: product_name, quantity, urgency_level, supplier_name
  - Missing fields: Flags required parameters not provided by the user

Example Input: "We're almost out of 500ml bottles, maybe 20 boxes"

Parsed Output:

```JSON
{
  "intent": "restock_request",
  "product_name": "500ml bottles",
```

```
  "requested_quantity": 20,
  "unit": "boxes",
  "urgency": "medium",
  "missing_fields": ["current_stock", "supplier_preference"]
}
```

- Multi-step Reasoning: GLM decides reorder quantity, urgency, and supplier based on:
  1. System rules: Stock threshold, approval limits (RM 10,000)
  2. Context: Current stock level from `needsRestock()` API
  3. Supplier data: Rating, lead time, price from `getBestSupplier()`

Reasoning Chain Example:

```
None
Step 1: Check stock via needsRestock(product_id=1)
  → Current stock: 5 units | Threshold: 10 units
  → Decision: NEEDS_RESTOCK = true

Step 2: Calculate reorder quantity
  → Formula: (Threshold × 2) - Current Stock = (10 × 2) - 5 = 15 units
  → User requested 20 units → Take max(15, 20) = 20 units

Step 3: Select supplier via getBestSupplier(product_id=1)
  → Supplier A: RM45, rating 4.8, lead time 2 days
  → Supplier B: RM42, rating 4.2, lead time 5 days
  → Decision: Supplier A (faster delivery + higher reliability)

Step 4: Check approval threshold
  → 20 units × RM45 = RM900 (< RM10,000)
  → Decision: NO_APPROVAL_NEEDED

Step 5: Execute via restock(product_id=1)
  → Order created successfully
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

- Dynamic Tool Orchestration: GLM selects and invokes the correct endpoint based on reasoning output. The tool mapping is defined in the function calling schema:

Tool Name	API Endpoint	When Called
`check_stock`	`GET /inventory/restock-check`	Before any restock decision
`update_stock`	`POST /inventory/add or /remove`	After sales confirmation or restock receipt
`list_suppliers`	`GET /suppliers/{productId}`	When the user asks, "Who supplies this?"
`get_best_supplier`	`GET /suppliers/{productId}/best`	For optimal supplier selection
`trigger_restock`	`POST /procurement/restock/{productId}`	After approval or when urgent
`request_approval`	Custom (WhatsApp/Email)	When order > RM10,000
`create_purchase_order`	`POST /orders/create`	After approval received

- Failure Handling:
  - Level 1 - Missing Data (User Clarification)

```
None
User: "Order more rice."
GLM: "How many kg? Current stock unknown. Please specify quantity or type 'check stock' to see current levels."
```

  - Level 2 - API Failure (Retry with Backoff)

```
None
Attempt 1: call getBestSupplier(product_id=1) → TIMEOUT (5s)
Wait 1 second
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

```
Attempt 2: call getBestSupplier(product_id=1) → TIMEOUT (5s)
Wait 2 seconds
Attempt 3: call getBestSupplier(product_id=1) → TIMEOUT (5s)
→ Escalate to Level 3
```

- Level 3 - Human Escalation

```
None
GLM Response: "⚠️ Supplier API is currently unavailable after 3 attempts.
I've notified your procurement manager via WhatsApp.
Please try again in 5 minutes or approve manual override.
Context: Restock request for 20 units of Product #1 (rice bags)"
```

- Structured Outputs: Generates machine-readable outputs that integrate with existing entities:
  - Purchase Order (maps to `Order` entity)

```JSON
{
  "id": "PO-001",
  "productId": 1,
  "supplierId": 3,
  "quantity": 20,
  "status": "PENDING",
  "totalValue": 900.00,
  "createdAt": "2026-04-25T10:30:00Z"
}
```

  - Inventory Log (maps to `InventoryLog` entity)

```JSON
{
  "productId": 1,
  "change": -5,
  "timestamp": 1745577000,
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

```
  "reason": "customer_order_#ORD-42",
  "performedBy": "system_auto"
}
```

- Dashboard Notification

```JSON
{
  "type": "LOW_STOCK_ALERT",
  "productId": 1,
  "productName": "Rice Bags",
  "currentStock": 5,
  "threshold": 10,
  "actionRequired": true,
  "suggestedRestock": 20
}
```

4.3. AI Model & Prompt Design

4.3.1. Model Selection:

Z. AI's GLM is used as the sole reasoning engine.

Justification against requirements:

Requirement	GLM Capability	How It Maps to InvAgent
Function calling	GLM can decide which tool to invoke based on natural language	Decides between `check_stock`, `create_po`, `send_approval` based on user intent
Long context	Handles lengthy email threads or inventory logs	Processes 8K token email threads containing order history + supplier negotiations

© UMHackathon 2026

Email: umhackathon@um.edu.my

Reasoning under uncertainty	Handles ambiguous requests like "maybe restock soon."	Classifies "maybe restock soon" as LOW urgency, asks clarifying questions, or schedules a check
Statefulness	Maintains workflow state across turns	Tracks `awaiting_approval` state until the manager responds via WhatsApp
Multi-API orchestration	Sequences multiple tool calls	Executes `check_stock` → `get_best_supplier` → `request_approval` → `trigger_restock` in correct order

4.3.2. Prompting Strategy:

Multi-step agentic prompting with the ReAct (Reason + Act) pattern.

- Why:
  - Inventory workflows require iterative reasoning: check stock → decide → act → observe stock level → next decision
  - Zero-shot prompting fails for multi-API orchestration because GLM cannot remember which APIs were already called
  - ReAct provides an explicit "Thought → Action → Observation" loop that mirrors human procurement logic
- Complete prompt template:

None

```
┌──────────────────────────────────────────────────────────────┐
│                     SYSTEM PROMPT (Fixed)                    |
├──────────────────────────────────────────────────────────────┤
| You are InvAgent, an inventory procurement assistant for     |
| Malaysian SMEs. You have access to these tools:              |
|                                                              |
| TOOLS:                                                       |
| 1. check_stock(product_name or product_id) → returns {      |
|      current_stock: int, threshold: int, needs_restock: bool |
|    }                                                         |
| 2. list_suppliers(product_id) → returns [{supplier_id, name, |
|      price, rating, lead_time}]                              |
| 3. get_best_supplier(product_id) → returns {supplier_id,     |
|      name, price, rating, lead_time, reason}                 |
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

```
| 4. request_approval(order_details) → sends WhatsApp/email      |
| 5. trigger_restock(product_id, quantity, supplier_id) → calls  |
|    POST /procurement/restock/{productId}                       |
| 6. update_stock(product_id, quantity, action: add/remove) →    |
|    calls POST /inventory/add or /remove                        |
 |                                                                |
 | RULES:                                                         |
 | - Always check stock before suggesting restock quantity       |
 | - Total order value = quantity × supplier_price               |
 | - If total > RM10,000: MUST call request_approval before      |
 |   trigger_restock                                              |
 | - If user doesn't provide quantity: calculate as              |
 |   max(user_suggestion, (threshold × 2) - current_stock)       |
 | - If any API fails 3 times: escalate with full context        |
 | - Respond in Bahasa Malaysia or English based on user's       |
 |   language                                                     |
 | - Urgency keywords: "urgent"/"ASAP"=HIGH, "low"/"running      |
 |   out"=MEDIUM, "maybe"/"eventually"=LOW                        |
 └──────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────┐
|                     WORKFLOW STATE (Dynamic)                   |
├──────────────────────────────────────────────────────────────┤
 | Current State: {                                               |
 |   "step": "awaiting_stock_check|awaiting_supplier|awaiting_    |
 |            approval|completed|failed",                        |
 |   "product_id": null,                                          |
 |   "product_name": "rice bags",                                 |
 |   "requested_quantity": 20,                                    |
 |   "calculated_quantity": null,                                 |
 |   "urgency": "HIGH",                                           |
 |   "selected_supplier": null,                                   |
 |   "approval_status": null,                                     |
 |   "failure_count": 0,                                          |
 |   "last_error": null                                           |
 | }                                                              |
 └──────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────┐
|                     USER INPUT (Variable)                      |
├──────────────────────────────────────────────────────────────┤
 | User: "Need more rice urgently. Maybe 20 bags."                |
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

```
└──────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────┐
│                     REASONING OUTPUT                      │
 ├──────────────────────────────────────────────────────────┤
 │ Thought 1: User wants rice bags, quantity 20, urgency HIGH.   │
 │            Need to check current stock first per rule 1.      │
 │ Action 1: check_stock(product_name="rice bags")               │
 │                                                               │
 │ Observation 1: {current_stock: 5, threshold: 10,              │
 │                 needs_restock: true}                          │
 │                                                               │
 │ Thought 2: Stock is 5 (below threshold 10). Needs restock.    │
 │            Calculate quantity: max(20, (10×2)-5)=max(20,15)=20│
 │            Need best supplier.                                │
 │ Action 2: get_best_supplier(product_id=1)                     │
 │                                                               │
 │ Observation 2: {supplier_id: 3, name: "Beras Wangi",          │
 │                 price: 45, rating: 4.8, lead_time: 2}         │
 │                                                               │
 │ Thought 3: Total = 20 × RM45 = RM900 (< RM10,000). No         │
 │            approval needed. Trigger restock.                  │
 │ Action 3: trigger_restock(product_id=1, quantity=20,          │
 │            supplier_id=3)                                     │
 │                                                               │
 │ Final Answer: "✅ Restocked 20 bags of rice from Beras Wangi. │
 │                Order #PO-001 created. Expected delivery: 2    │
 │                days."                                         │
 └──────────────────────────────────────────────────────────┘
```

4.3.3. Context & Input Handling:

- Handling unstructured inputs: GLM extracts intent, entities (item name, quantity, urgency), and missing fields.
- Few-shot extraction examples embedded in the prompt:

```
None
Example 1:
Input: "We're out of cooking oil, need 10 bottles ASAP"
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

```
Extracted: {intent: "restock", product: "cooking oil", quantity: 10, urgency:
"HIGH"}

Example 2:
Input: "Maybe restock rice if we're low"
Extracted: {intent: "check_and_restock", product: "rice", urgency: "LOW", missing:
["quantity"]}

Example 3:
Input: "Who supplies our flour?"
Extracted: {intent: "supplier_query", product: "flour"}
```

- Oversized inputs (e.g., long email threads):

Input Size	Strategy
< 8K tokens	Direct processing
8K - 32K tokens	Chunking with overlap (500 token overlap between 4K chunks) → Summarization prompt per chunk → Recombine summaries
> 32K tokens	Rejection: "Input too large. Please summarize or upload as a file."

Chunking example for 15K token email thread:

```
None
Chunk 1 (0-4K): Header + first email → Summary: "Initial request for 50 units"
Chunk 2 (3.5K-7.5K): Reply chain → Summary: "Supplier quoted RM45/unit,
negotiation ongoing"
Chunk 3 (7K-11K): Follow-up → Summary: "User agreed to RM45, requested expedited
shipping"
Chunk 4 (10.5K-15K): Final confirmation → Summary: "Order confirmed, waiting for
PO creation"

Combined summary → 200 tokens passed to GLM instead of 15K
```

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.3.4. Fallback & Failure Behavior:

- Hallucination / off-topic response:
  - Detect via confidence score + schema validation (e.g., output must contain action field).
  - Retry with a lower temperature (0.2) and a more constrained prompt.
  - After 2 failures → Graceful error state: "I'm unsure. Please rephrase or approve manual override."
- Human escalation path: For critical actions (e.g., large POs >RM 10k), GLM pauses and requests manager approval via WhatsApp.

5. User Stories & Use Cases

- User Stories:
  - *"As a product manager, I want to enter a prompt using my own way of describing so that it provides me with a design without manual drawing by me."*
  - *"As a procurement manager, I want GLM to ask for approval only when the order value exceeds the limit, not for every small item."*

- Use Cases (Main Interactions):
  - **Voice-to-Procurement:**
    User: "We're low on 500ml bottles, maybe 20 boxes."
    GLM: checks stock (10 boxes left), calculates reorder (20 boxes), lists suppliers, requests approval, and creates a PO.
  - **Missing Data Handling:**
    User: "Order more rice."
    GLM: "How many kg? Current stock unknown. Please specify or allow me to check the physical inventory system."
  - **Failure Recovery:**
    Supplier API fails → GLM logs error, tries secondary supplier, if still fails → notifies human with context.

- Use Case Diagram (Textual) :

[User] → (speak/type request) → [InvAgent Interface]
[InvAgent] → calls [GLM Reasoner]
[GLM] → invokes [Stock API], [Supplier API], [Approval Queue], [PO System]
[GLM] ← returns status → [User]
If failure → [Human Approver] (escalation path)

© UMHackathon 2026

6. Features Included (Scope Definition)

- Natural language intake (text & voice via STT)
- Multi-step procurement workflow orchestration
- Dynamic tool/API calling based on GLM decisions
- Stateful session memory (e.g., awaiting approval)
- Automatic PO generation (structured output)
- Fallback & retry logic for hallucinations
- Human escalation for high-value or failed actions

7. Features Not Included (Scope Control)

- No physical robotics or IoT integration (initial version)
- No multi-warehouse conflict resolution (future)
- No predictive analytics for demand forecasting (pure workflow focus)
- No direct payment processing

8. Assumptions & Constraints

- **LLM Cost Constraint:**
  Estimated 0.5–2 cents per user session (average 2K input tokens + 1K output tokens).
  Design decision to reduce cost:
  - Caching frequent queries: “Check stock of rice” results are cached for 5 minutes.
  - Compressed state representation: Instead of full chat history, GLM receives a compact workflow state JSON (e.g., {step: "awaiting_approval", item: "rice", qty: 50}).
- **Technical Constraints:**
  PO creation only via API to existing accounting system (no native invoice generation).
  Requires internet for GLM API calls.
- **Performance Constraints:**
  Each workflow step adds 2–5 seconds of latency due to GLM reasoning + API calls.
  High-priority items get priority in the queue.
- **User Input:**
  The system cannot act without human initiation. All final POs require explicit human approval (configurable threshold).

9. Risks & Questions Throughout Development

- Workflow Drift:
  How to ensure GLM follows the correct procurement policy?
  → Use few-shot examples in the system prompt showing correct step sequences. Validate outputs against a state machine.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- API Failure Cascades:
  What if the stock API fails mid-workflow?
  → GLM marks the step as failed, saves the state, and retries with exponential backoff. After 3 failures, it escalates to a human with full context.
- Ambiguity in Urgency:
  User says "kind of urgent" – how to decide?
  → GLM is prompted to classify urgency as low/medium/high based on keywords. If uncertain, ask user: "On a scale of 1-5, how urgent?"
- Data Privacy:
  Inventory levels and supplier lists are sensitive.
  → All GLM prompts are stripped of PII (supplier names hashed). No logging of raw inputs beyond workflow state.

© UMHackathon 2026

Email: umhackathon@um.edu.my